

TariffRadar

Detailed Descriptions of All Figures

Chapters 2, 3 & 4 — English

This document provides comprehensive English-language descriptions of all 28 figures from Chapters 2, 3, and 4 of the TariffRadar diploma project. Each entry includes the figure number, descriptive title, the original image, and a structured breakdown of all visual elements, components, and technical details shown in the diagram.

System: TariffRadar — B2B SaaS logistics tariff monitoring platform (Belarus/CIS market).
Stack: Go 1.21 · PostgreSQL 15 · Redis 7 · Docker · JWT · REST API · OpenAPI 3.0

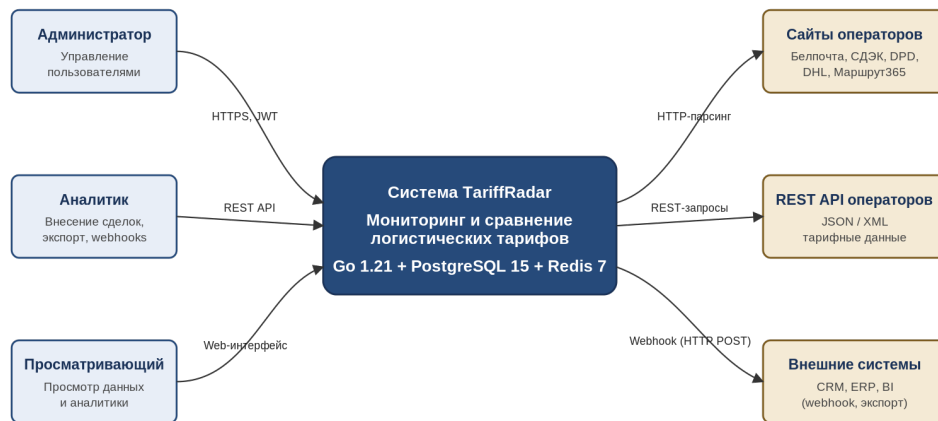
Chapter 2 — System Architecture Design (10 figures)

Chapter 3 — Go Server-Side Development (10 figures)

Chapter 4 — Database and Comparison Module (8 figures)

CHAPTER 2 — System Architecture Design

Figure 2.1
TariffRadar System — Context Diagram (C4 Level 1)



This is a high-level context diagram that shows the TariffRadar system as a central black box and all external actors or systems that interact with it. The diagram follows the C4 model Level 1 convention, providing a bird's-eye view of the system's boundaries.

Central System:

- TariffRadar — labeled as 'Monitoring and Comparison of Logistics Tariffs', built on Go 1.21 + PostgreSQL 15 + Redis 7 (dark navy rounded rectangle).

User Roles (left side, three blue boxes):

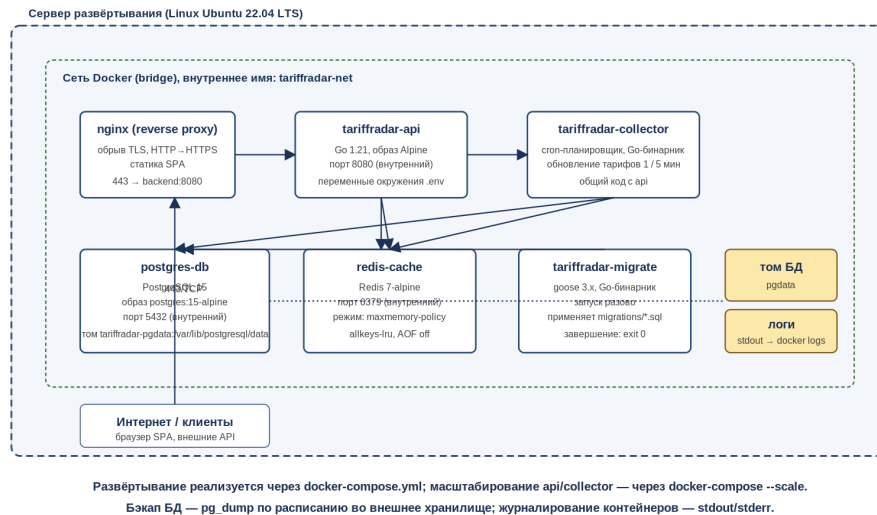
- Administrator — manages users, connects via HTTPS/JWT.
- Analyst — submits deals, exports data, manages webhooks, connects via REST API.
- Viewer — reads market data and analytics, connects via Web interface.

External Systems (right side, three beige boxes):

- Operator Websites — Belpochta, SDEK, DPD, DHL, Marshrut365; connected via HTTP parsing (web scraping).
- REST API of Operators — delivers JSON/XML tariff data via direct REST requests.
- External Systems — CRM, ERP, BI tools receive data via Webhook HTTP POST notifications.

The diagram illustrates the complete trust boundary of the system: all three user roles push data IN, while the system pushes notifications OUT to external integrations and pulls tariff data from operator sources.

Figure 2.2
Docker Deployment Architecture — Containers and Network



This deployment diagram shows how TariffRadar is containerized and deployed on a single Linux Ubuntu 22.04 LTS server. All containers share a custom Docker bridge network named 'tariffadar-net', which provides internal DNS resolution and network isolation.

Containers inside the Docker network:

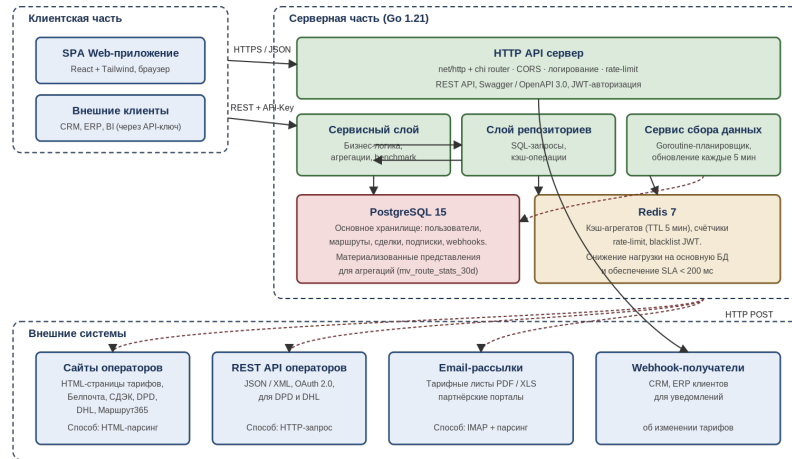
- `nginx` (reverse proxy) — terminates TLS, forwards HTTP to HTTPS, serves the SPA static files, routes port 443 to backend:8080.
- `tariffadar-api` — the main Go 1.21 application binary on Alpine Linux, listens on internal port 8080, reads config from `.env` environment variables.
- `tariffadar-collector` — cron-scheduled Go binary that updates tariffs every 5 minutes; shares source code with the API container via multi-stage build.
- `postgres-db` — PostgreSQL 15-alpine, port 5432 (internal only), data persisted to a named volume 'pgdata' mapped to `/var/lib/postgresql/data`.
- `redis-cache` — Redis 7-alpine, port 6379 (internal only), `maxmemory-policy allkeys-lru`, AOF disabled.
- `tariffadar-migrate` — one-shot container running `goose 3.x` to apply all pending migrations/`/*.sql` files, exits with code 0 on success.

Persistent volumes (yellow boxes on the right):

- Database volume (`pgdata`) — persists PostgreSQL data across container restarts.
- Logs — all `stdout/stderr` flows to `docker logs` for centralized log collection.

Internet clients (browser SPA, external APIs) connect at the bottom through `nginx`. Deployment is managed by `docker-compose.yml`; horizontal scaling of `api/collector` is possible via `docker-compose --scale`. Database backups are scheduled via `pg_dump` to external storage.

Figure 2.3
Full System Architecture — Layered Component View



This is a detailed layered architecture diagram showing all major components of the TariffRadar system and how they relate to each other, from client interfaces down to external data sources.

Client tier:

- SPA Web Application (React + Tailwind) and External Clients (CRM, ERP, BI) connect via HTTPS/JSON to the HTTP API Server.

HTTP API Server (Go 1.21):

- Uses net/http with chi router, Swagger/OpenAPI 3.0 documentation, JWT authentication, CORS.

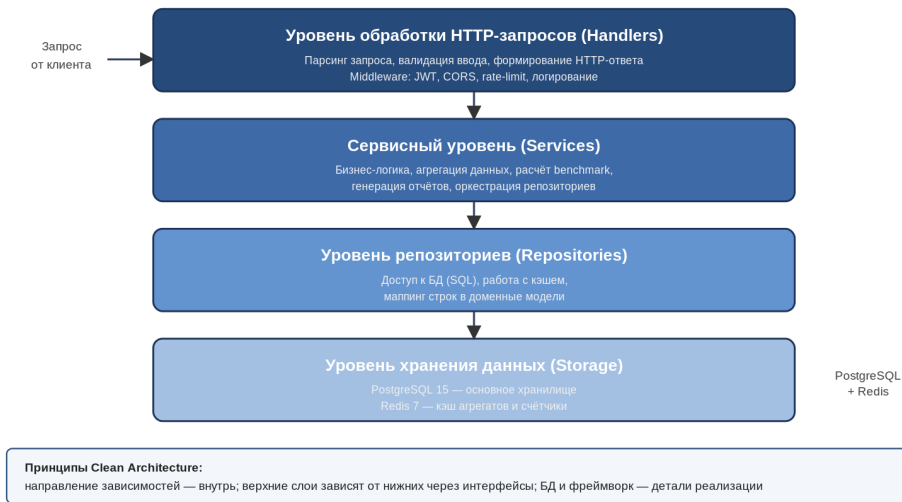
Internal layers (top-down):

- Service Layer — implements business logic: data aggregation, benchmark calculation, report generation, orchestrates repositories.
- Repository Layer — abstracts data access: SQL queries, Redis cache operations, row-to-domain-model mapping.
- Storage — PostgreSQL 15 (tables: users, routes, deals, plans, subscriptions, materialized views) and Redis 7 (cache TTL keys, rate limit counters, JWT blacklist).

External data sources (bottom tier):

- Operator Websites — HTML parsing via golang.org/x/net/html.
- REST API of Operators (DPD, DHL) — JSON/XML responses.
- Email notifications — PDF/XLS report distribution.
- Webhook Recipients — receive HTTP POST on price.changed events.

Figure 2.4
Clean Architecture — Four-Layer Dependency Model



This diagram illustrates the application of Clean Architecture principles in TariffRadar, showing four strictly ordered horizontal layers with a single direction of dependency (always pointing inward/downward). A client request enters at the top and data flows through each layer sequentially.

Layer 1 — HTTP Request Handlers (top, dark navy):

- Responsibilities: parse incoming requests, validate input, format HTTP responses.
- Middleware chain: JWT authentication, CORS headers, rate limiting, structured logging.

Layer 2 — Service Layer (medium navy):

- Responsibilities: business logic, data aggregation, benchmark calculations, report generation, repository orchestration.

Layer 3 — Repository Layer (medium-light navy):

- Responsibilities: SQL query execution, Redis cache read/write, mapping database rows to domain model structs.

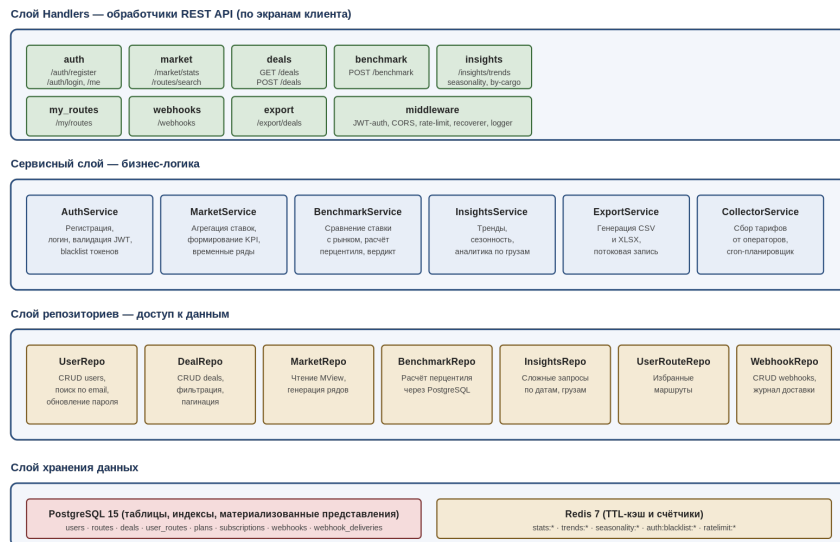
Layer 4 — Storage Layer (light blue, right side labels: PostgreSQL + Redis):

- PostgreSQL 15 — primary persistent store for all business data.
- Redis 7 — cache for aggregated data and rate-limit counters.

Clean Architecture principle (note box at bottom):

- Dependency direction is always inward: upper layers depend on lower layers only through interfaces.
- Database and framework are treated as implementation details, not core domain concerns.

Figure 2.5
Full Component Decomposition Across All Four Layers



This is the most detailed component diagram in Chapter 2, listing every individual Go package, handler, service, and repository in the system, grouped by architectural layer. It serves as a complete inventory of the application's internal structure.

Handlers layer (8 handler groups + middleware):

- auth — /auth/register, /auth/login, /me
- market — /market/stats, /routes/search
- deals — GET /deals (list), POST /deals (add)
- benchmark — POST /benchmark (rate comparison)
- insights — /insights/trends, /seasonality, /by-cargo
- my_routes — /my/routes (saved routes)
- webhooks — /webhooks CRUD
- export — /export/deals (CSV/XLSX download)
- middleware — JWT-auth, CORS, rate-limit, panic recoverer, request logger

Service layer (6 services):

- AuthService — registration, login, JWT validation, token blacklisting
- MarketService — rate aggregation, KPI calculation, time series
- BenchmarkService — user rate vs. market comparison, percentile
- InsightsService — trends, seasonality patterns, cargo-type analytics
- ExportService — CSV/XLSX streaming generation
- CollectorService — tariff collection from operators, cron scheduler

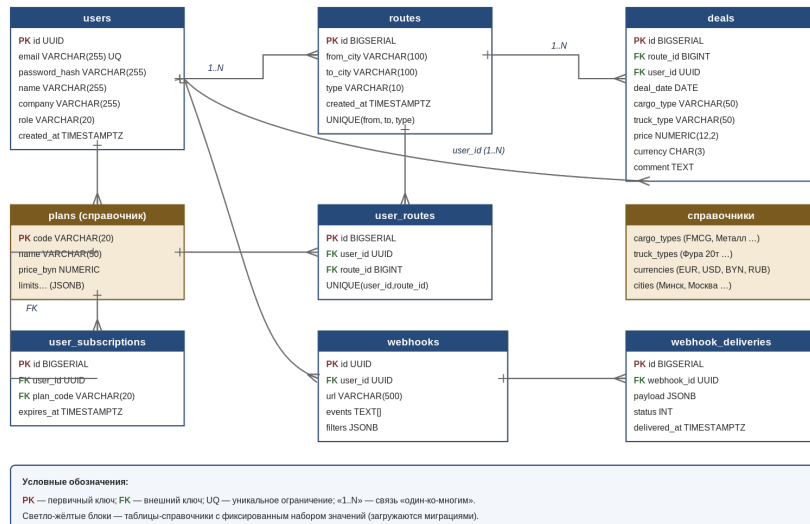
Repository layer (7 repositories):

- UserRepo, DealRepo, MarketRepo, BenchmarkRepo, InsightsRepo, UserRouteRepo, WebhookRepo

Storage layer:

- PostgreSQL 15 — tables: users, routes, deals, user_routes, plans, subscriptions, webhooks, webhook_deliveries
- Redis 7 — key namespaces: stats:*, trends:*, seasonality:*, auth:blacklist:*, ratelimit:*

Figure 2.6
Entity-Relationship Database Schema



This ER diagram shows the complete relational schema of the TariffRadar PostgreSQL database, including all tables, their columns with data types, primary/foreign keys, constraints, and materialized views. The schema is designed for multi-tenant B2B SaaS use.

Core tables:

- **users** — id UUID PK (gen_random_uuid), email VARCHAR UNIQUE, password_hash VARCHAR, role VARCHAR CHECK IN ('viewer', 'analyst', 'admin'), company VARCHAR, name VARCHAR, created_at TIMESTAMPTZ.
- **routes** — id BIGSERIAL PK, from_city VARCHAR, to_city VARCHAR, type VARCHAR, UNIQUE(from_city, to_city, type).
- **deals** (main fact table) — id BIGSERIAL PK, route_id FK, user_id FK, price NUMERIC(12,2), cargo_type VARCHAR, truck_type VARCHAR, currency CHAR(3) CHECK, deal_date DATE, comment TEXT. Partitioned by RANGE(deal_date) in the scaling plan.

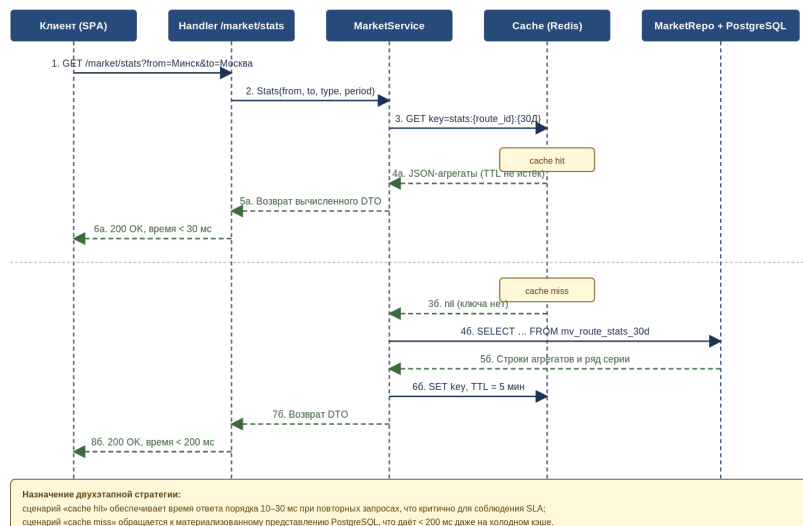
Supporting tables:

- **plans** — subscription plan definitions (code, name, limits).
- **user_subscriptions** — user_id FK, plan_code VARCHAR, expires_at TIMESTAMPTZ.
- **user_routes** — user_id FK, route_id FK (saved/favorited routes).
- **webhooks** — id UUID PK, user_id FK, url VARCHAR, events JSONB (array of event types), filter JSONB.
- **webhook_deliveries** — id BIGSERIAL, webhook_id FK, payload JSONB, response_status INT, attempt INT, delivered_at TIMESTAMPTZ.

Materialized views (shown in green):

- **mv_route_stats_30d** — pre-computed AVG/MIN/MAX/COUNT per route for last 30 days, refreshed CONCURRENTLY every 5 minutes.
- **mv_route_series_30d** — time-series pre-aggregation for chart rendering.

Figure 2.7
Sequence Diagram — GET /market/stats: Cache Hit vs. Cache Miss



This UML sequence diagram shows the complete request lifecycle for the most frequently called endpoint — GET /market/stats — illustrating two distinct execution paths: a fast cache hit path and a slower cache miss path that falls back to PostgreSQL.

Actors (5 vertical swimlanes):

- Client (SPA), Handler /market/stats, MarketService, Cache (Redis), MarketRepo + PostgreSQL

Cache HIT scenario (steps 1–6a, upper portion):

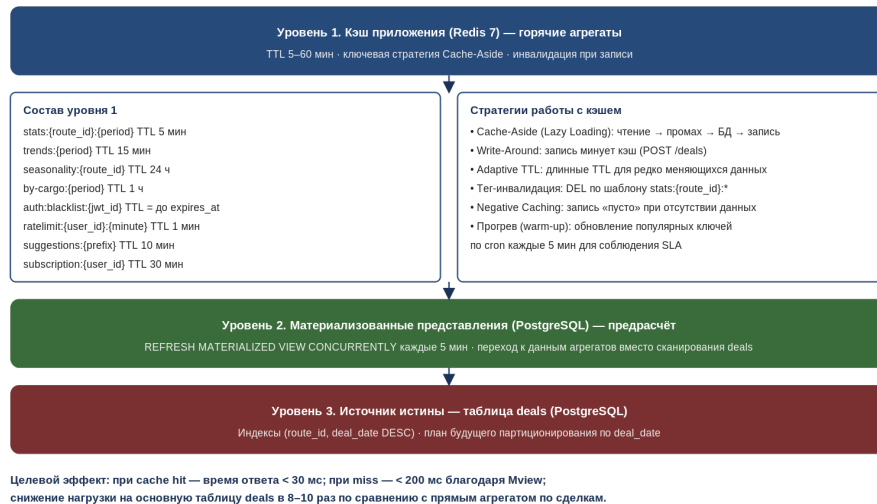
- 1. Client sends GET /market/stats?from=Minsk&to=Moscow
- 2. Handler calls MarketService.Stats(from, to, type, period)
- 3. Service calls Redis GET key=stats:{route_id}:{30d}
- 4a. Redis returns JSON aggregates (TTL not expired — cache hit)
- 5a. Service returns computed DTO to Handler
- 6a. Handler responds 200 OK — total time < 30 ms

Cache MISS scenario (steps 1–8b, lower portion):

- 3b. Redis returns nil (key absent)
- 4b. MarketRepo executes SELECT ... FROM mv_route_stats_30d
- 5b. PostgreSQL returns aggregate rows and series data
- 6b. Service calls Redis SET key with TTL = 5 minutes
- 7b. DTO returned to Handler
- 8b. 200 OK — total time < 200 ms

The two-phase strategy ensures <30ms on repeat requests (cache hit) and <200ms even on cold cache by reading from the materialized view rather than scanning all deals rows.

Figure 2.8
Three-Tier Caching Architecture



This diagram describes the layered caching strategy used in TariffRadar to achieve sub-200ms API response times. Three tiers are stacked vertically, each providing progressively slower but more authoritative data.

Tier 1 — Redis 7 Application Cache (top, dark navy):

- Hot aggregates with TTL ranging from 1 minute to 24 hours; Cache-Aside strategy with invalidation-on-write.
- Key namespaces: stats:{route_id}:{period} (5 min), trends:{period} (15 min), seasonality:{route_id} (24 h), by-cargo:{period} (1 h), auth:blacklist:{jwt_id} (until expires_at), ratelimit:{user_id}:{minute} (1 min), suggestions:{prefix} (10 min), subscription:{user_id} (30 min).
- Cache strategies: Cache-Aside (Lazy Loading), Write-Around (POST /deals bypasses cache), Adaptive TTL (longer TTL for rarely-changing data), Tag-invalidation (DEL stats:{route_id}:* pattern), Negative Caching (empty response cached to prevent thundering herd), Warm-up (popular keys refreshed by cron every 5 min to maintain SLA).

Tier 2 — PostgreSQL Materialized Views (middle, dark green):

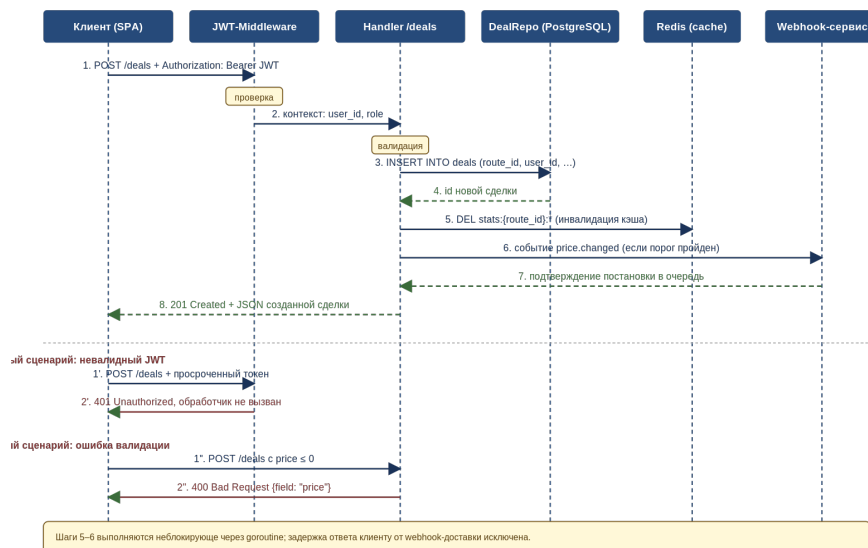
- Pre-computed aggregates refreshed with REFRESH MATERIALIZED VIEW CONCURRENTLY every 5 minutes.
- Transition from real-time deals aggregation to reading pre-computed rows.

Tier 3 — deals table (bottom, dark red):

- Source of truth. B-Tree indexes on (route_id, deal_date DESC). Future partitioning plan by deal_date.

Target performance: cache hit < 30 ms; cache miss < 200 ms thanks to MView; load on the deals table reduced 8-10x compared to direct aggregation queries.

Figure 2.9
Sequence Diagram — POST /deals: Deal Submission with Cache Invalidation



This sequence diagram documents the full lifecycle of a POST /deals request — the primary data ingestion endpoint. It covers the happy path, JWT error handling, and input validation failure scenarios across 6 swimlane participants.

Participants:

- Client (SPA), JWT-Middleware, Handler /deals, DealRepo (PostgreSQL), Redis (cache), Webhook-service

Happy path (steps 1–8):

- 1. Client sends POST /deals with Authorization: Bearer JWT token and deal JSON body.
- 2. JWT-Middleware validates token, injects user_id and role into request context.
- 3. Handler validates input, then calls DealRepo: INSERT INTO deals (route_id, user_id, ...).
- 4. PostgreSQL returns the id of the newly created deal.
- 5. Handler calls Redis DEL stats:{route_id}:* to invalidate all cached aggregates for this route.
- 6. If price change exceeds configured threshold, a price.changed event is queued to WebhookSvc (non-blocking goroutine).
- 7. WebhookSvc confirms event queued.
- 8. Handler responds 201 Created with the created deal JSON.

Error scenario 1 — Invalid JWT:

- 1'. POST /deals with expired token.
- 2'. JWT-Middleware returns 401 Unauthorized; handler never invoked.

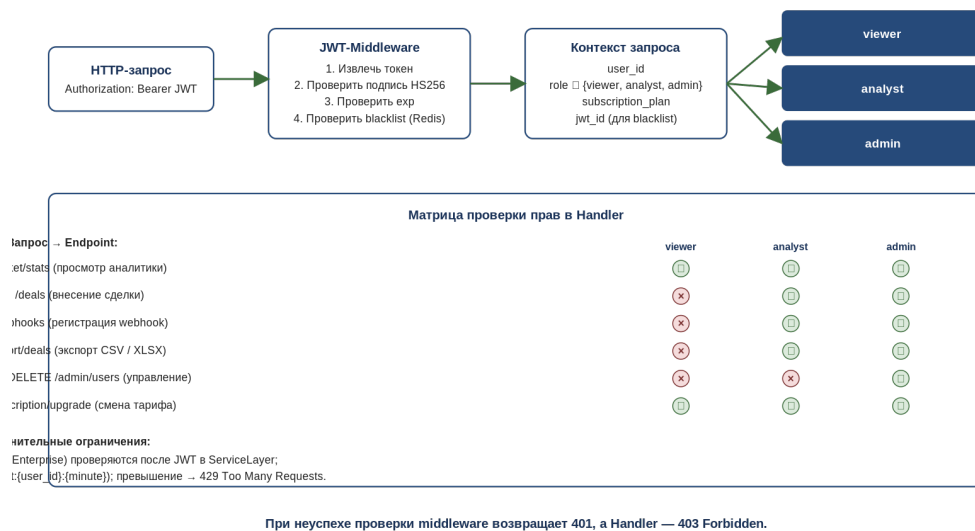
Error scenario 2 — Validation failure:

- 1''. POST /deals with price <= 0.
- 2''. Handler returns 400 Bad Request with field: 'price'.

Steps 5 and 6 execute non-blocking in a goroutine, so webhook delivery latency does not affect the response time returned to the client.

Figure 2.10

JWT Authentication Middleware — Verification Flow and Role Permission Matrix



This combined diagram has two parts: a flow diagram showing the JWT verification pipeline, and a permissions matrix table showing which API endpoints each role is allowed to access.

JWT Verification Pipeline (top, 4 boxes):

- HTTP Request arrives with Authorization: Bearer JWT header.
- JWT-Middleware executes 4 sequential checks: (1) extract the token string, (2) verify HS256 HMAC signature, (3) verify the exp (expiry) claim, (4) check the jwt_id against Redis blacklist (for logout support).
- On success: injects into Request Context — user_id, role (viewer/analyst/admin), subscription_plan, jwt_id.
- Routes handler to one of three role branches: viewer, analyst, admin.

Permission Matrix (bottom table — 6 endpoints × 3 roles):

- /market/stats (view analytics) — allowed for ALL roles: viewer, analyst, admin.
- POST /deals (submit deal) — DENIED for viewer; allowed for analyst, admin.
- POST /webhooks (register webhook) — DENIED for viewer; allowed for analyst, admin.
- GET /export/deals (CSV/XLSX export) — DENIED for viewer; allowed for analyst, admin.
- DELETE /admin/users (user management) — DENIED for viewer AND analyst; admin ONLY.
- /subscription/upgrade (change plan) — allowed for ALL roles.

Additional constraints:

- Subscription-plan limits (e.g., Enterprise-only features) are checked after JWT in the ServiceLayer.
- Rate limiting: ratelimit:{user_id}:{minute} in Redis; exceeding limit returns 429 Too Many Requests.

On verification failure: middleware returns 401 Unauthorized. On role check failure inside a handler: returns 403 Forbidden.

CHAPTER 3 — Go Server-Side Development

Figure 3.1
TariffRadar Go Project Directory Structure

```

tariffadar/
cmd/
  server/
    main.go — точка входа, инициализация и graceful shutdown
internal/
  config/
    config.go — загрузка и валидация конфигурации
    config_test.go — unit-тесты конфига
  handlers/ — HTTP-обработчики (по экранам клиента)
    auth.go market.go deals.go benchmark.go
    insights.go my_routes.go webhooks.go export.go
    response.go — writeJSON / writeError / readJSON
    response_test.go
  middleware/
    auth.go — JWT-middleware, контекст user_id и role
  service/ — бизнес-логика
    auth.go market.go benchmark.go
    insights.go export.go auth_test.go
  repository/ — доступ к данным
    postgres.go redis.go cache.go
    users.go market.go deals.go benchmark.go
    insights.go user_routes.go webhooks.go
  models/
    models.go — User, Route, Deal, Role, Currency и т.д.
migrations/ — SQL-миграции (формат goose)
  0001_init.sql users + trigger updated_at
  0002_routes_and_deals.sql routes, deals, индексы, справочники
  0003_user_routes.sql избранные маршруты
  0004_webhooks.sql webhooks + webhook_deliveries
docs/
  openapi.yaml — спецификация REST API (OpenAPI 3.0)
  swagger-ui.html — интерфейс просмотра API
docker-compose.yml Dockerfile Makefile .env.example go.mod go.sum

```

This is an ASCII-art tree diagram of the complete Go project directory structure for TariffRadar. It shows the physical layout of source files, following Go's idiomatic 'cmd/internal' layout convention for production applications.

Top-level directories and files:

- cmd/server/main.go — application entry point: initialization sequence, environment loading, graceful shutdown handler.
- internal/ — all private application packages (not importable by external code).

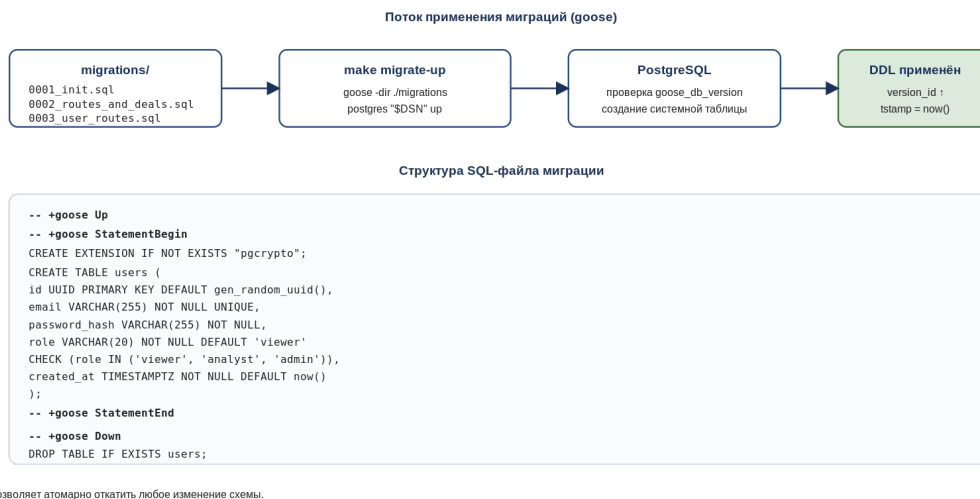
internal/ subdirectories:

- config/config.go — loads configuration from .env file and environment variables into a Config struct.
- handlers/ — HTTP handler files: auth.go, market.go, deals.go, benchmark.go, insights.go, my_routes.go, response.go (JSON write helpers).
- middleware/auth.go — JWT authentication middleware; extracts user_id and role into context.
- service/ — business logic: auth.go, market.go, benchmark.go, export.go, insights.go.
- repository/ — data access: postgres.go (pgxpool setup), redis.go (go-redis client), cache.go (GetOrSet helper), users.go, market.go, insights.go, webhooks.go.
- models/models.go — domain type definitions: Deal, Route, Role, Currency, and DTOs.

Project root files:

- migrations/ — numbered SQL migration files (0001_init.sql, 0002_routes_and_deals.sql, 0003_user_routes.sql, 0004_webhooks.sql) managed by goose.
- docs/swagger.yaml — OpenAPI 3.0 specification auto-generated by swaggo/swag.
- docker-compose.yml, Dockerfile, Makefile, .env.example, go.mod, go.sum.

Figure 3.2
Goose Database Migration Flow and SQL File Structure



This figure has two parts: a process flow showing how goose applies database migrations, and a code example showing the internal structure of a migration SQL file.

Migration application flow (top, 4 steps):

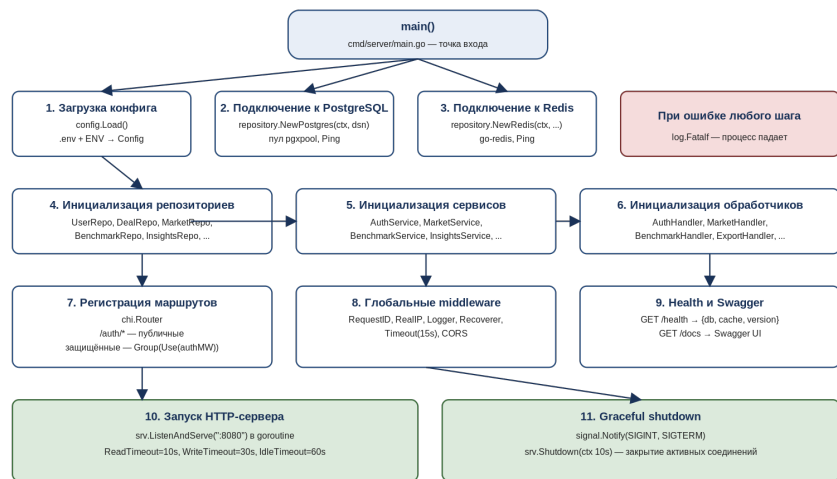
- 1. migrations/ folder — contains numbered SQL files: 0001_init.sql, 0002_routes_and_deals.sql, 0003_user_routes.sql.
- 2. make migrate-up — invokes: `goose -dir ./migrations postgres "$DSN" up`
- 3. PostgreSQL — goose checks the `goose_db_version` system table to determine which migrations have already been applied; creates the tracking table if absent.
- 4. DDL applied — `version_id` incremented, `tstamp = now()` recorded.

SQL migration file structure (bottom code block):

- `-- +goose Up` / `-- +goose StatementBegin` — marks the beginning of the upgrade block.
- `CREATE EXTENSION IF NOT EXISTS 'pgcrypto'` — enables UUID generation.
- `CREATE TABLE users (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), email VARCHAR(255) NOT NULL UNIQUE, password_hash VARCHAR(255) NOT NULL, role VARCHAR(20) NOT NULL DEFAULT 'viewer' CHECK (role IN ('viewer','analyst','admin')), created_at TIMESTAMPTZ NOT NULL DEFAULT now());`
- `-- +goose StatementEnd` marks the end of the transactional block.
- `-- +goose Down` / `DROP TABLE IF EXISTS users` — rollback script.

Each migration is wrapped in a transaction, allowing atomic rollback of any schema change if an error occurs mid-migration. Goose ensures idempotency: already-applied migrations are skipped.

Figure 3.3
main() Initialization Sequence — 11-Step Application Bootstrap



Все ошибки шагов 1–3 фатальны: процесс не должен ст
с дефолтным секретом или недоступной БД.

This flowchart documents all 11 sequential initialization steps that occur in `cmd/server/main.go` before the HTTP server begins accepting requests. Steps 1-3 are critical (fatal on failure); subsequent steps wire up the dependency injection chain.

Steps 1–3 (infrastructure connections, any failure = `log.Fatalf`):

- 1. Load Config — `config.Load()` reads `.env` file and ENV variables into a typed Config struct.
- 2. Connect to PostgreSQL — `repository.NewPostgres(ctx, dsn)` creates a `pgxpool` connection pool; calls `Ping()` to verify connectivity.
- 3. Connect to Redis — `repository.NewRedis(ctx, ...)` initializes `go-redis` client; calls `Ping()`.
- Error handler (red box): any failure in steps 1-3 causes `log.Fatalf` — the process terminates.

Steps 4–6 (dependency injection chain):

- 4. Initialize Repositories — `UserRepo`, `DealRepo`, `MarketRepo`, `BenchmarkRepo`, `InsightsRepo`, and others are instantiated with database connections.
- 5. Initialize Services — `AuthService`, `MarketService`, `BenchmarkService`, `InsightsService` receive their repository dependencies via constructor injection.
- 6. Initialize Handlers — `AuthHandler`, `MarketHandler`, `BenchmarkHandler`, `ExportHandler` receive service dependencies.

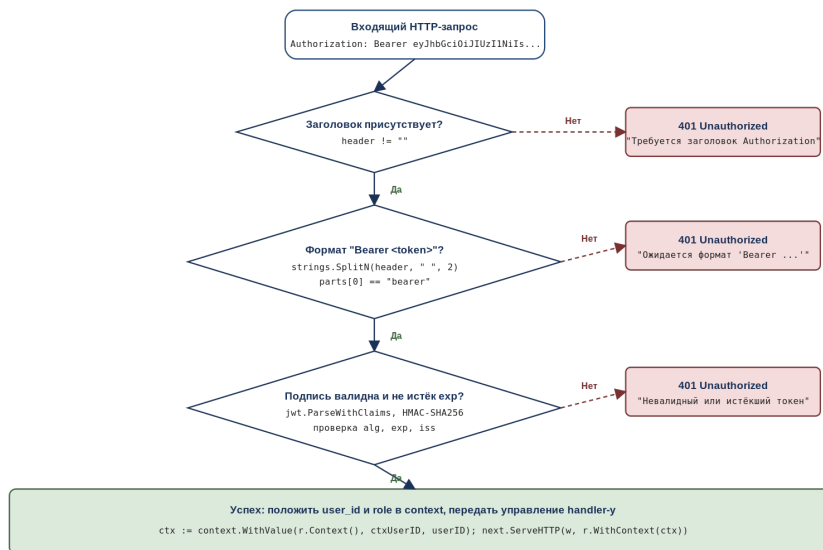
Steps 7–9 (HTTP server configuration):

- 7. Register Routes — `chi.Router` created; `/auth/*` routes are public; protected routes wrapped in `Group(Use(authMW))`.
- 8. Global Middleware — `RequestID`, `RealIP`, `Logger`, `Recoverer` (panic → 500), `Timeout(15s)`, `CORS`.
- 9. Health and Swagger — `GET /health` returns `{db, cache, version}`; `GET /docs` serves Swagger UI.

Steps 10–11 (server lifecycle):

- 10. Start HTTP Server — `srv.ListenAndServe("8080")` launched in a goroutine; `ReadTimeout=10s`, `WriteTimeout=30s`, `IdleTimeout=60s`.
- 11. Graceful Shutdown — `signal.Notify` listens for `SIGINT/SIGTERM`; on signal calls `srv.Shutdown(ctx 10s)` to drain active connections before exit.

Figure 3.4
JWT Middleware — Token Validation Flowchart



This flowchart details the step-by-step JWT token validation logic executed by the authentication middleware for every request to a protected endpoint. It shows three sequential decision gates, each with a specific 401 error response on failure.

Entry point:

- Incoming HTTP request with Authorization: Bearer eyJhbGciOiJIUzI1NiIs... header.

Decision Gate 1 — Header present?

- Check: Authorization header != empty string.
- Failure (No): returns 401 Unauthorized — 'Authorization header required'.

Decision Gate 2 — Format 'Bearer '?

- Check: strings.SplitN(header, ' ', 2); parts[0] == 'bearer' (case-insensitive).
- Failure (No): returns 401 Unauthorized — 'Expected format Bearer ...'

Decision Gate 3 — Signature valid and exp not expired?

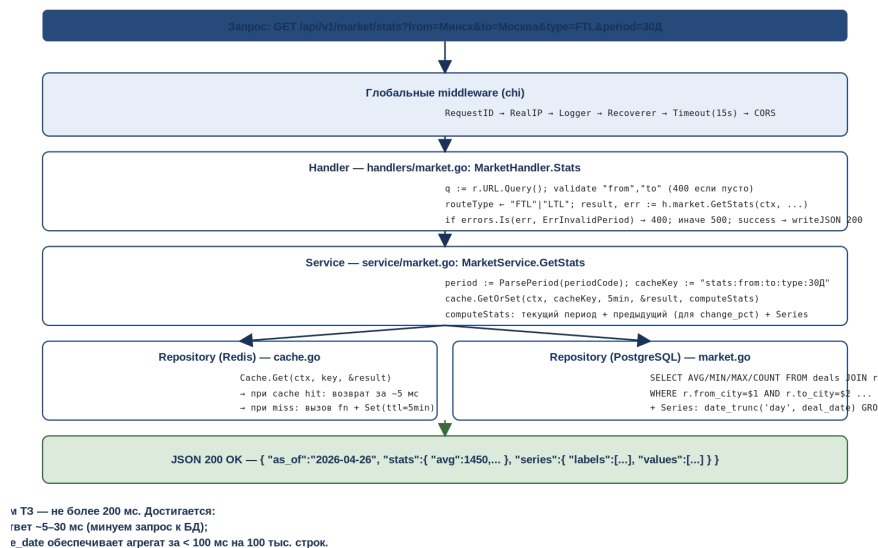
- Check: jwt.ParseWithClaims using HMAC-SHA256 secret; verifies alg, exp, iss claims.
- Also checks Redis blacklist for the jwt_id (supports server-side logout).
- Failure (No): returns 401 Unauthorized — 'Invalid or expired token'.

Success path:

- ctx := context.WithValue(r.Context(), ctxUserID, userID) — injects user_id into context.
- next.ServeHTTP(w, r.WithContext(ctx)) — passes enriched context to the next handler.

This middleware is registered as a chi route group middleware and never executes on public routes (/auth/register, /auth/login).

Figure 3.5
GET /market/stats — Full Request Lifecycle Across All Layers



This vertical flow diagram traces a single GET /api/v1/market/stats request from network entry to JSON response, showing the exact code path through each architectural layer with actual function signatures and SQL snippets.

Request entry (dark navy header):

- GET /api/v1/market/stats?from=Minsk&to=Moscow&type=FTL&period=30D

Layer 1 — Global Middleware (chi chain):

- RequestID → RealIP → Logger → Recoverer → Timeout(15s) → CORS

Layer 2 — Handler (handlers/market.go: MarketHandler.Stats):

- `q := r.URL.Query(); validate 'from','to' (400 if empty).`
- `routeType` defaults to 'FTL'/'LTL'; calls `h.market.GetStats(ctx, ...)`.
- Error mapping: `errors.Is(err, ErrInvalidPeriod) → 400`; otherwise 500; success → `writeJSON 200`.

Layer 3 — Service (service/market.go: MarketService.GetStats):

- `period := ParsePeriod(periodCode); cacheKey := 'stats:from:to:type:30D'.`
- `cache.GetOrSet(ctx, cacheKey, 5min, &result, computeStats).`
- `computeStats` calculates current period + previous (for `change_pct`) + Series data.

Layer 4 — Repositories (two parallel paths):

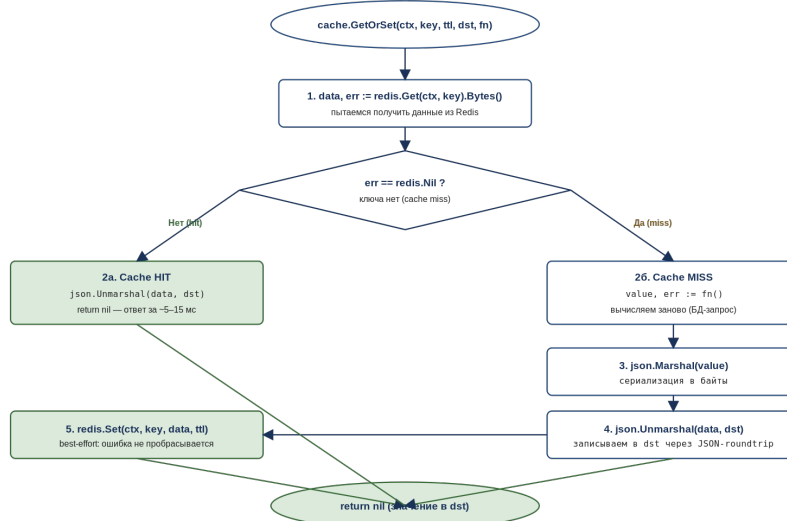
- Repository Redis (cache.go): `Cache.Get` returns ~5ms on hit, calls `fn + Set` on miss.
- Repository PostgreSQL (market.go): `SELECT AVG/MIN/MAX/COUNT FROM deals JOIN routes WHERE r.from_city=$1 AND r.to_city=$2...; + Series via date_trunc('day', deal_date) GROUP BY.`

Response:

- JSON 200 OK: { "as_of": "2026-04-26", "stats": { "avg": 1450, ... }, "series": { "labels": [...], "values": [...] } }

SLA requirement from TZ: ≤ 200 ms. Achieved: cache hit ~5–30ms (no DB query); date_trunc on deal_date index delivers aggregation in < 100ms on 100k rows.

Figure 3.6
cache.GetOrSet() — Cache-Aside Algorithm Flowchart



This flowchart shows the internal logic of the `cache.GetOrSet()` helper function, which implements the Cache-Aside (Lazy Loading) pattern used throughout the application to transparently cache expensive database computations.

Function signature:

- `cache.GetOrSet(ctx, key, ttl, dst, fn)` — `ctx`: request context; `key`: Redis key string; `ttl`: expiry duration; `dst`: pointer to result struct; `fn`: function that computes the value from DB.

Step 1 — Attempt Redis read:

- `data, err := redis.Get(ctx, key).Bytes()` — tries to fetch cached bytes.

Decision — `err == redis.Nil` (key absent)?

- No (key found — Cache HIT path, left branch): 2a. `json.Unmarshal(data, dst)` — deserialize cached JSON into `dst` struct; return `nil`. Time: ~5-15ms.
- Yes (key absent — Cache MISS path, right branch): 2b. `value, err := fn()` — calls the provided compute function (database query). If `fn` errors, returns error immediately.

Cache MISS continuation:

- 3. `json.Marshal(value)` — serialize the computed value to JSON bytes.
- 4. `json.Unmarshal(data, dst)` — write value into `dst` via JSON round-trip (type-safe copy).
- 5. `redis.Set(ctx, key, data, ttl)` — store in Redis; best-effort (error not propagated to caller).

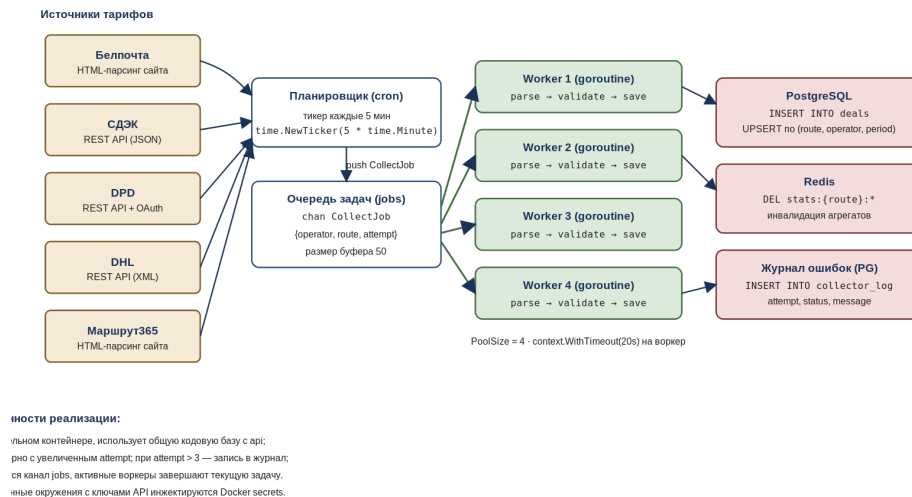
Convergence:

- Both paths arrive at 'return nil (value in dst)' — caller always receives the result in `dst`.

The best-effort Redis write in step 5 ensures a Redis outage doesn't break the request — the system gracefully degrades to direct database queries.

Figure 3.7

Data Collector Architecture — Concurrent Worker Pool with Operator Adapters



This architecture diagram shows the TariffRadar data collection subsystem, which automatically harvests tariff data from 5 logistics operator sources every 5 minutes using a concurrent worker pool pattern.

Data Sources (left, 5 boxes):

- Belpochta — HTML website scraping (golang.org/x/net/html parser).
- SDEK — REST API, JSON response format.
- DPD — REST API, OAuth 2.0 authentication.
- DHL — REST API, XML response format.
- Marshrut365 — HTML website scraping.

Scheduler (center-top):

- cron ticker: `time.NewTicker(5 * time.Minute)` — fires every 5 minutes.
- On each tick: generates `CollectJob` structs for all operator+route combinations.
- Jobs sent to: `chan CollectJob` with buffer size 50 (prevents scheduler blocking).

Worker Pool (4 goroutines):

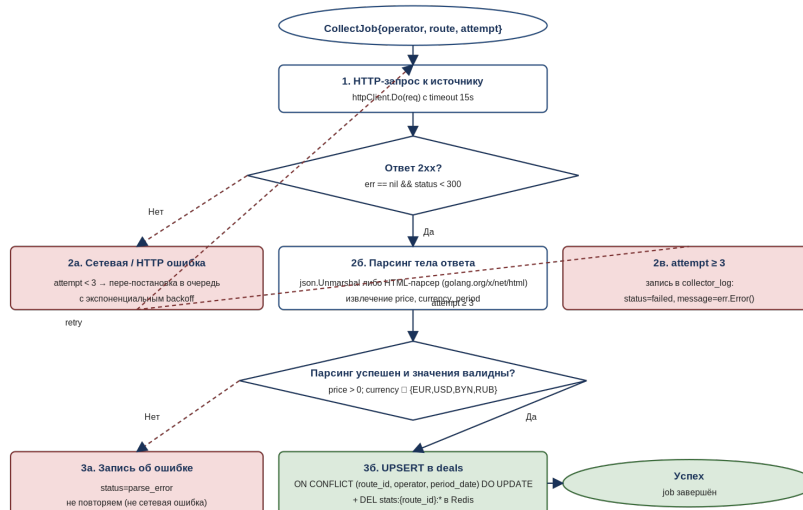
- 4 concurrent workers process jobs from the channel simultaneously.
- Each worker executes the pipeline: `parse` → `validate` → `save`.
- Pool size = `context.WithTimeout(20s)` per job to prevent hanging workers.

Outputs (right side):

- PostgreSQL — `INSERT INTO deals / UPSERT ON CONFLICT (route_id, operator, period_date) DO UPDATE`.
- Redis — `DEL stats:{route_id}:*` to invalidate cached aggregates for updated routes.
- Error log — `INSERT INTO collector_log: attempt number, status (failed/parse_error), message`.

Figure 3.8

CollectJob Execution Flowchart — HTTP Fetch, Parse, and UPSERT with Retry



This flowchart details the complete execution logic for a single `CollectJob`, showing how the collector handles network errors with exponential backoff retry, parse failures, and successful data persistence.

Entry:

- `CollectJob{operator, route, attempt}` — where `attempt` starts at 1.

Step 1 — HTTP Request:

- `httpClient.Do(req)` with 15-second timeout.

Decision Gate 1 — Response 2xx?

- No (network or HTTP error): if `attempt < 3`, re-queue job with exponential backoff delay (attempt 1→1min, attempt 2→5min); if `attempt >= 3`, write to `collector_log` with `status=failed`.
- Yes: proceed to step 2b.

Step 2b — Parse Response Body:

- JSON: `json.Unmarshal(body, &dto;)` for REST API operators.
- HTML: `golang.org/x/net/html` tokenizer for website scraping.
- Extract: `price` (float), `currency` (string), `period` (date).

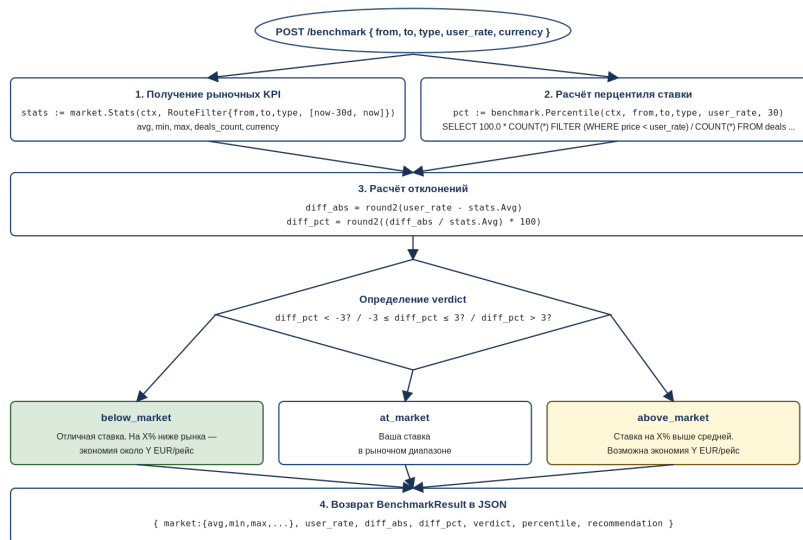
Decision Gate 2 — Parse successful and values valid?

- Valid: `price > 0`; currency must be in `{EUR, USD, BYN, RUB}`.
- No (parse error): write `collector_log` `status=parse_error`; do NOT retry (not a transient error).

Step 3b — UPSERT in deals table:

- `INSERT INTO deals (...) ON CONFLICT (route_id, operator, period_date) DO UPDATE SET price=EXCLUDED.price.`
- `+ Redis DEL stats:{route_id}:*` to trigger cache refresh on next request.
- → Job completed successfully.

Figure 3.9
POST /benchmark — Benchmark Calculation Algorithm



This flowchart describes the complete algorithm for the benchmark endpoint, which compares a user's freight rate against the current market and returns a verdict with savings recommendations.

Entry:

- POST /benchmark with body: {from, to, type, user_rate, currency}

Two parallel operations (steps 1 and 2):

- 1. Get Market KPIs — calls `market.Stats(ctx, RouteFilter{from,to,type,[now-30d,now]})` to retrieve avg, min, max, deals_count, currency for the requested route.
- 2. Calculate Percentile — calls `benchmark.Percentile(ctx, from, to, type, user_rate, 30)`; executes: `SELECT 100.0 * COUNT(*) FILTER (WHERE price < user_rate) / COUNT(*) FROM deals JOIN routes WHERE ...` — returns what percentage of deals are cheaper than user_rate.

Step 3 — Calculate Deviations:

- `diff_abs = round2(user_rate - stats.Avg)`
- `diff_pct = round2((diff_abs / stats.Avg) * 100)`

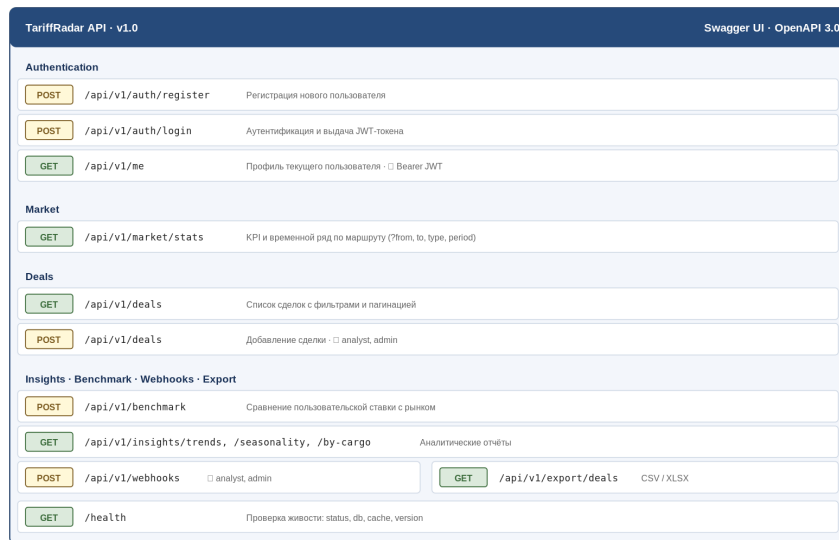
Step 4 — Determine Verdict (three-way branch):

- below_market: `diff_pct < -3%` → 'Excellent rate, X% below market — savings of ~Y EUR/trip'.
- at_market: `-3% <= diff_pct <= 3%` → 'Your rate is within market range'.
- above_market: `diff_pct > 3%` → 'Rate X% above market average. Potential savings: Y EUR/trip'.

Step 5 — Return BenchmarkResult JSON:

- {market:{avg,min,max,...}, user_rate, diff_abs, diff_pct, verdict, percentile, recommendation}

Figure 3.10
Swagger UI — TariffRadar API v1.0 Endpoint Reference



This screenshot-style mockup shows the Swagger UI interface for the TariffRadar REST API documentation, served at GET /docs and generated from the OpenAPI 3.0 specification. It provides an interactive interface for exploring and testing all API endpoints.

Header:

- Title: 'TariffRadar API · v1.0', tag: 'Swagger UI · OpenAPI 3.0' (dark navy bar).

Authentication section (3 endpoints):

- POST /api/v1/auth/register — user registration (orange POST badge).
- POST /api/v1/auth/login — authentication and JWT token issuance.
- GET /api/v1/me — current user profile; requires Bearer JWT (green GET badge).

Market section (1 endpoint):

- GET /api/v1/market/stats — KPI and time series by route; query parameters: ?from=, ?to=, ?type=, ?period=.

Deals section (2 endpoints):

- GET /api/v1/deals — paginated list of deals with filters.
- POST /api/v1/deals — add a new deal; requires analyst or admin role.

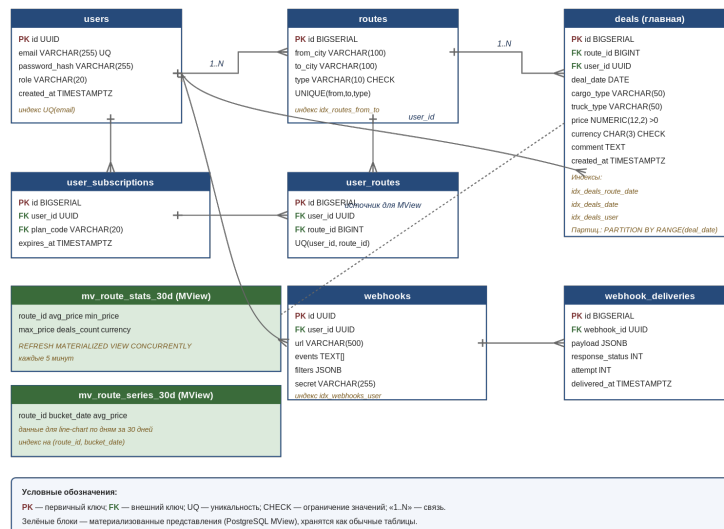
Insights · Benchmark · Webhooks · Export section (5 endpoints):

- POST /api/v1/benchmark — compare user rate with market.
- GET /api/v1/insights/trends, /seasonality, /by-cargo — analytics reports.
- POST /api/v1/webhooks — register webhook; requires analyst or admin role.
- GET /api/v1/export/deals — download CSV or XLSX file.
- GET /health — liveness probe returning {status, db, cache, version}.

CHAPTER 4 — Database and Comparison Module

Figure 4.1

Complete PostgreSQL Database ER Diagram with Materialized Views



This is the authoritative entity-relationship diagram for the full TariffRadar PostgreSQL schema, showing all tables with their columns, data types, constraints, relationships, and two materialized views used for performance optimization.

Core tables (top row):

- users — id UUID PK (gen_random_uuid), email VARCHAR(255) UNIQUE, password_hash VARCHAR(255), role VARCHAR CHECK, company VARCHAR(255), name VARCHAR(255), created_at TIMESTAMPTZ.
- routes — id BIGSERIAL PK, from_city VARCHAR(100), to_city VARCHAR(100), type VARCHAR(10), UNIQUE(from_city, to_city, type).
- deals (main fact table) — id BIGSERIAL PK, route_id BIGINT FK, user_id UUID FK, price NUMERIC(12,2), cargo_type VARCHAR(50), truck_type VARCHAR(50), currency CHAR(3) CHECK, deal_date DATE, comment TEXT. PARTITION BY RANGE(deal_date) scaling plan shown.

Supporting tables:

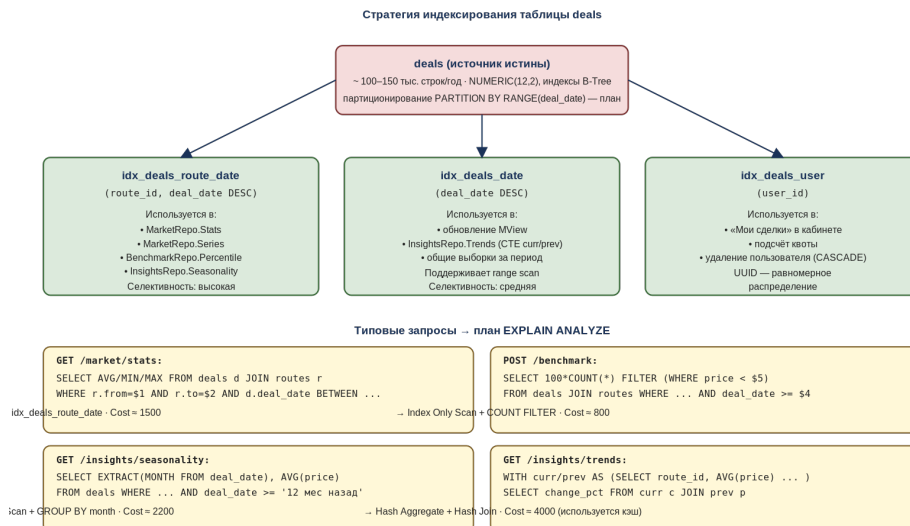
- user_subscriptions — FK to user_id, plan_code VARCHAR, expires_at TIMESTAMPTZ.
- user_routes — FK to user_id and route_id (many-to-many saved routes).
- webhooks — id UUID PK, user_id FK, url VARCHAR(500), events JSONB, filter JSONB.
- webhook_deliveries — id BIGSERIAL, webhook_id FK, payload JSONB, response_status INT, attempt INT, delivered_at TIMESTAMPTZ.

Materialized views (green tables):

- mv_route_stats_30d — stores AVG/MIN/MAX/COUNT per route for the last 30 days; refreshed CONCURRENTLY every 5 minutes by cron.
- mv_route_series_30d — time-series pre-aggregation for chart rendering.

Legend: PK = primary key, FK = foreign key with CASCADE, 1:N = one-to-many relationship. Green tables = materialized views (PostgreSQL MView), not regular tables.

Figure 4.2
Indexing Strategy for the deals Table — Three B-Tree Indexes with Query Plans



This diagram documents the indexing strategy for the deals table — the most heavily queried table in the system — showing three B-Tree indexes, their purposes, selectivity, and the EXPLAIN ANALYZE cost estimates for typical queries.

Central: deals table characteristics:

- ~100-150k rows per year growth; NUMERIC(12,2) price column; future PARTITION BY RANGE(deal_date).

Index 1 — idx_deals_route_date (route_id, deal_date DESC):

- Used by: MarketRepo.Stats, MarketRepo.Series, BenchmarkRepo.Percentile, InsightsRepo.Seasonality.
- Selectivity: HIGH — route_id is a foreign key with limited distinct values combined with date range filtering.

Index 2 — idx_deals_date (deal_date DESC):

- Used by: MView REFRESH (CONCURRENTLY), InsightsRepo.Trends (CTE curr/prev queries), general period range queries.
- Supports efficient range scan; medium selectivity.

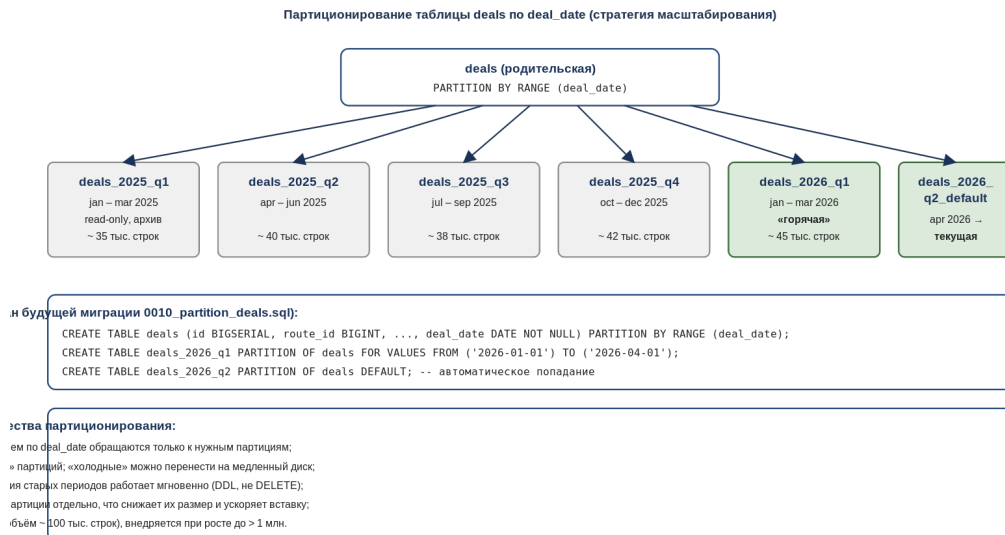
Index 3 — idx_deals_user (user_id):

- Used by: 'My Deals' user dashboard, quota counting, CASCADE user deletion.
- UUID primary key provides uniform hash distribution.

4 sample query EXPLAIN ANALYZE plans:

- GET /market/stats — Index Scan on idx_deals_route_date, estimated Cost ≈ 1500.
- POST /benchmark — Index Only Scan + COUNT FILTER, Cost ≈ 800.
- GET /insights/seasonality — Index Scan + GROUP BY month, Cost ≈ 2200.
- GET /insights/trends — Hash Aggregate + Hash Join via CTE, Cost ≈ 4000 (uses Redis cache).

Figure 4.3
deals Table Partitioning Strategy — Quarterly Range Partitions



This diagram illustrates the PostgreSQL table partitioning design for the deals table, showing how data is split into quarterly range partitions to support efficient archival, query pruning, and long-term scalability beyond 1 million rows.

Parent table:

- deals — PARTITION BY RANGE(deal_date) — the parent table contains no rows directly; all data goes to child partitions.

6 child partitions (shown as boxes with arrows from parent):

- deals_2025_q1 — January–March 2025, read-only archive, ~35,000 rows.
- deals_2025_q2 — April–June 2025, ~40,000 rows.
- deals_2025_q3 — July–September 2025, ~38,000 rows.
- deals_2025_q4 — October–December 2025, ~42,000 rows.
- deals_2026_q1 — January–March 2026 (highlighted as 'hot'), ~45,000 rows.
- deals_2026_q2_default — April 2026 and beyond, current active partition.

SQL migration snippet (0010_partition_deals.sql):

- CREATE TABLE deals (... , deal_date DATE NOT NULL) PARTITION BY RANGE(deal_date);
- CREATE TABLE deals_2026_q1 PARTITION OF deals FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');
- CREATE TABLE deals_2026_q2 PARTITION OF deals DEFAULT; -- catch-all for new data.

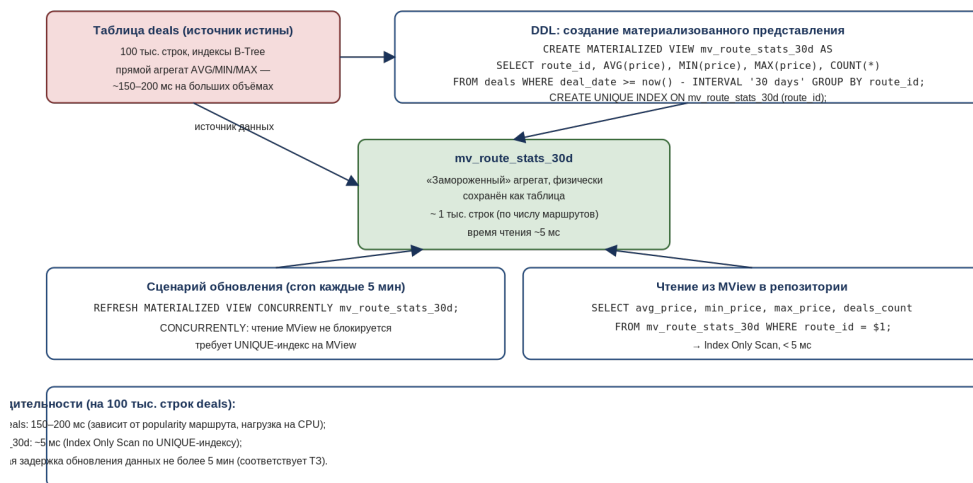
Partitioning benefits:

- Partition pruning: queries with deal_date filters scan only relevant partitions.
- Hot/cold tiering: 'cold' old partitions can be moved to slower/cheaper storage.
- Instant DDL archival: detaching old partitions is immediate, not DELETE.
- Separate index maintenance per partition reduces index size and insert cost.

Implemented at ~100k rows as preparation; target scale > 1 million rows by v2.0.

Figure 4.4

Materialized View mv_route_stats_30d — Lifecycle and Performance



This diagram shows the complete lifecycle of the most important PostgreSQL materialized view — mv_route_stats_30d — from its DDL creation through concurrent refresh scheduling to repository read access, with performance numbers.

Source: deals table (top-left, red box):

- ~100,000 rows, B-Tree indexes. Direct AVG/MIN/MAX aggregation: ~150–200ms on large datasets.

DDL Creation (top-right):

- CREATE MATERIALIZED VIEW mv_route_stats_30d AS SELECT route_id, AVG(price), MIN(price), MAX(price), COUNT(*) FROM deals WHERE deal_date >= now() - INTERVAL '30 days' GROUP BY route_id;
- CREATE UNIQUE INDEX ON mv_route_stats_30d (route_id); — required for CONCURRENTLY refresh.

mv_route_stats_30d (center, green box):

- 'Frozen' aggregate — physically stored on disk as a regular table.
- ~1,000 rows (one row per distinct route).
- Read time: ~5ms via Index Only Scan.

Update scenario (bottom-left, cron every 5 minutes):

- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_route_stats_30d;
- CONCURRENTLY: the MView remains readable during refresh (no read lock).
- Requires the UNIQUE index on route_id.

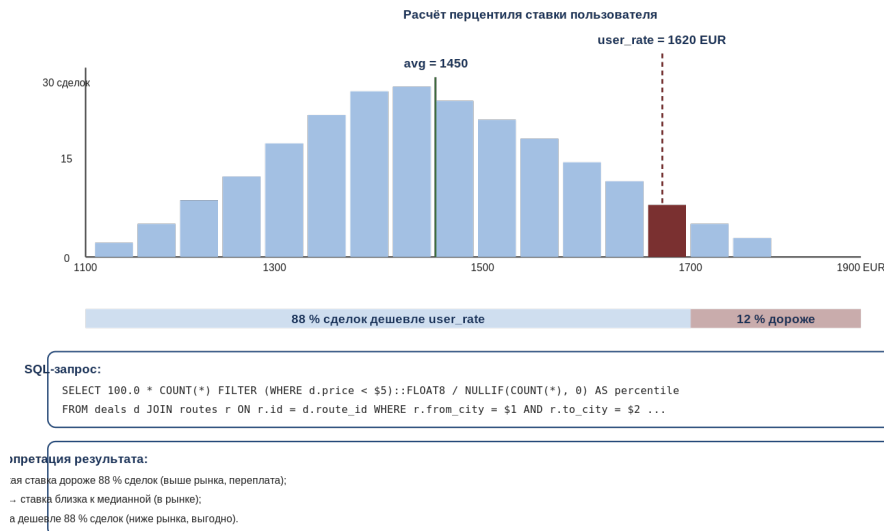
Repository read (bottom-right):

- SELECT avg_price, min_price, max_price, deals_count FROM mv_route_stats_30d WHERE route_id = \$1;
- → Index Only Scan — reads only the index, not the heap, < 5ms.

Performance comparison (bottom note):

- Direct deals query: 150–200ms (depends on route popularity, CPU load).
- MView read: ~5ms via Index Only Scan.
- Maximum data staleness: 5 minutes (matches TZ requirement).

Figure 4.5
Percentile Calculation — Price Distribution Histogram



This visualization illustrates how the benchmark percentile calculation works in practice, using a price distribution histogram to show where a user's rate falls relative to all market transactions on a given route.

Bar chart (top section):

- X-axis: deal prices in EUR, ranging from 1,100 to 1,900 EUR per route.
- Y-axis: number of deals (0 to 30 per price bucket).
- Blue bars: histogram of all anonymized market deal prices — bell-shaped distribution with peak around 1,400–1,500 EUR.
- Green vertical line at avg = 1,450 EUR — market average price.
- Dark red dashed vertical line + highlighted bar at user_rate = 1,620 EUR — the user's submitted rate, clearly above average.

Progress bar (middle):

- 88% of deals cheaper than user_rate (light blue portion).
- 12% more expensive than user_rate (pink portion).
- → The user's rate is at the 88th percentile — significantly above market.

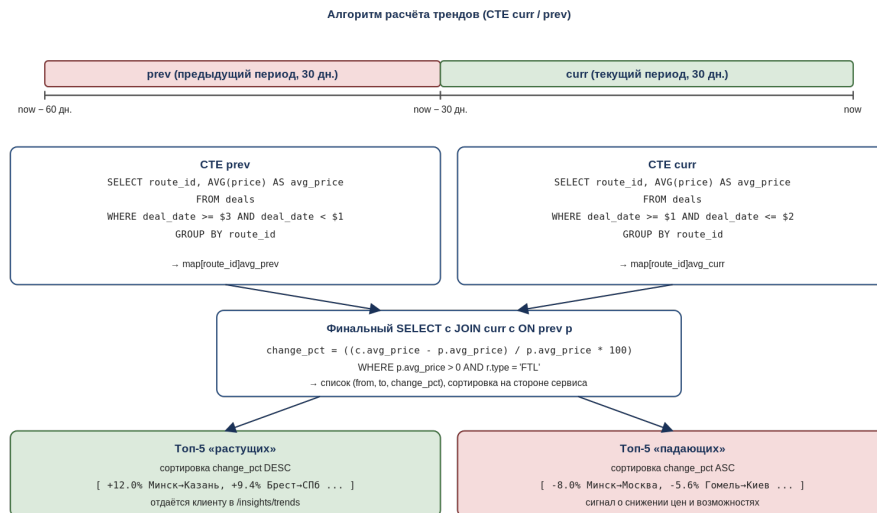
SQL query (bottom code block):

- `SELECT 100.0 * COUNT(*) FILTER (WHERE d.price < $5)::FLOAT8 / NULLIF(COUNT(*), 0) AS percentile`
- `FROM deals d JOIN routes r ON r.id = d.route_id WHERE r.from_city = $1 AND r.to_city = $2 ...`

Result interpretation:

- Percentile ~88%: user's rate is above 88% of market — overpaying.
- Percentile ~50%: rate is close to median — within market range.
- Percentile < 50%: rate is below market — competitive / favorable.

Figure 4.6
Trends Algorithm — CTE curr/prev Dual-Period Comparison



This diagram explains the SQL algorithm used to calculate route price trends by comparing two consecutive 30-day periods using Common Table Expressions (CTEs). It identifies the top growing and top falling routes for the Insights screen.

Timeline (top visual):

- Three time markers: now-60 days, now-30 days, now.
- prev period: from (now-60d) to (now-30d) — the reference baseline.
- curr period: from (now-30d) to (now) — the current window.

CTE prev (left box):

- `SELECT route_id, AVG(price) AS avg_price FROM deals WHERE deal_date >= $3 AND deal_date < $1 GROUP BY route_id`
- Result: `map[route_id]avg_prev`

CTE curr (right box):

- `SELECT route_id, AVG(price) AS avg_price FROM deals WHERE deal_date >= $1 AND deal_date <= $2 GROUP BY route_id`
- Result: `map[route_id]avg_curr`

Final SELECT with JOIN (center box):

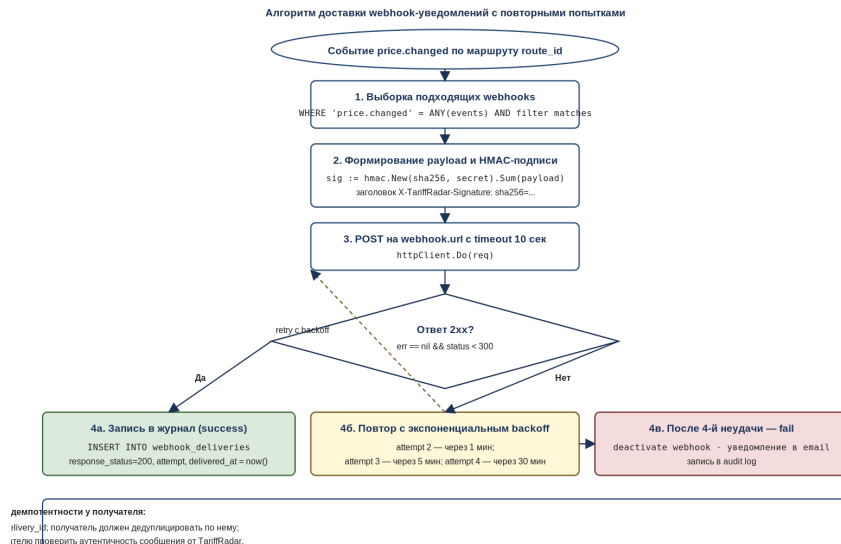
- $change_pct = ((c.avg_price - p.avg_price) / p.avg_price * 100)$
- Filter: `WHERE p.avg_price > 0 AND r.type = 'FTL'`
- Returns list of (from, to, change_pct), sorted server-side.

Output (two result boxes):

- Top-5 Growing routes (sort `change_pct DESC`): +12.0% Minsk→Kazan, +9.4% Brest→St.Petersburg...
- Top-5 Falling routes (sort `change_pct ASC`): -8.0% Minsk→Moscow, -5.6% Gomel→Kiev... — signals of price drops and opportunities.

Figure 4.7

Webhook Delivery Algorithm with Exponential Backoff Retry



This flowchart describes the complete webhook notification delivery pipeline, which runs asynchronously when the system detects a significant price change on a monitored route. It includes HMAC signing and exponential backoff retry logic.

Trigger:

- price.changed event fired for a specific route_id (triggered by DealService after deal insert).

Step 1 — Select matching webhooks:

- Query: WHERE 'price.changed' = ANY(events) AND filter JSON matches route criteria.

Step 2 — Build payload and HMAC signature:

- sig := hmac.New(sha256, secret).Sum(payload)
- Sets request header: X-TariffRadar-Signature: sha256=
- Recipients can verify the signature to confirm the message originated from TariffRadar.

Step 3 — HTTP POST to webhook.url:

- httpClient.Do(req) with 10-second timeout per delivery attempt.

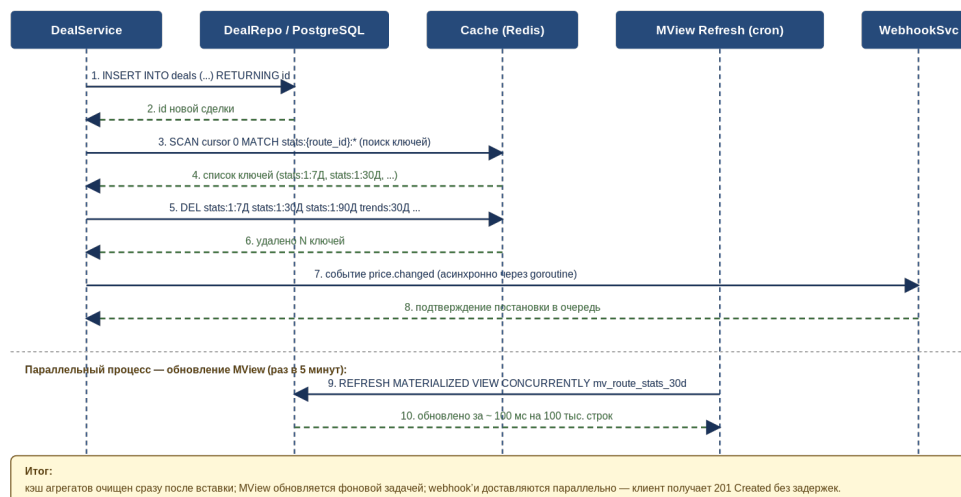
Decision — Response 2xx?

- Yes (success) → Step 4a: INSERT INTO webhook_deliveries (response_status=200, attempt, delivered_at=now()).
- No (failure) → Step 4b: Exponential backoff retry schedule: attempt 2 → 1 minute delay; attempt 3 → 5 minutes; attempt 4 → 30 minutes.
- After 4 failed attempts → Step 4c: deactivate webhook (set active=false), send email notification to webhook owner, write to audit log.

Idempotency note (bottom):

- Each delivery has a unique delivery_id; webhook receivers should deduplicate by it.
- Receivers must verify X-TariffRadar-Signature to authenticate the message origin.

Figure 4.8
Sequence Diagram — Data Consistency After Deal Insertion



This sequence diagram shows how the system maintains data consistency across PostgreSQL, Redis cache, the materialized view, and webhook delivery after a new deal is inserted — illustrating the complete side-effect chain of POST /deals.

Participants (5 swimlanes):

- DealService, DealRepo/PostgreSQL, Cache (Redis), MView Refresh (cron), WebhookSvc

Main flow (steps 1–8):

- 1. DealService calls DealRepo: INSERT INTO deals (...) RETURNING id.
- 2. DealRepo returns the new deal's id to DealService.
- 3. DealService calls Redis: SCAN cursor 0 MATCH stats:{route_id}:* to find all cached keys for this route.
- 4. Redis returns the list of matching keys (e.g., stats:1:7■, stats:1:30■, stats:1:90■).
- 5. DealService executes: DEL stats:1:7■ stats:1:30■ stats:1:90■ trends:30■ ... (batch delete all stale keys).
- 6. Redis confirms N keys deleted.
- 7. DealService fires price.changed event to WebhookSvc asynchronously (via goroutine — non-blocking).
- 8. WebhookSvc confirms event queued.

Parallel process — MView refresh (every 5 minutes by cron):

- 9. REFRESH MATERIALIZED VIEW CONCURRENTLY mv_route_stats_30d — runs independently.
- 10. MView updated in ~100ms on 100,000 rows; clients waiting for next API call will get fresh data.

Outcome summary (note box):

- Cache aggregates cleared immediately on insert.
- MView updated by background task — max 5-minute staleness.
- Webhooks delivered in parallel — client receives 201 Created without any delivery delay.